

Experiment 10- Use Ghidra to Disassemble and Analyze the Malware Code

Project Objectives:

- To provide a practical guide for disassembling and analyzing malware using Ghidra.
- To help users identify common malware functionalities such as persistence mechanisms, antianalysis methods, and network communication patterns.
- To train users to interpret low-level assembly instructions and correlate them with high-level **program behavior through Ghidra’s decompiler.**

Prerequisites:

- Ghidra: Installed and properly configured (requires Java Development Kit - JDK).
- Virtual Machine (VM): A secure and isolated setup such as VMware or VirtualBox to safely execute or inspect binaries.
- Sample Binary: A harmless or controlled executable or hex dump file for analysis.

Project Outline (GitHub Repository Components):

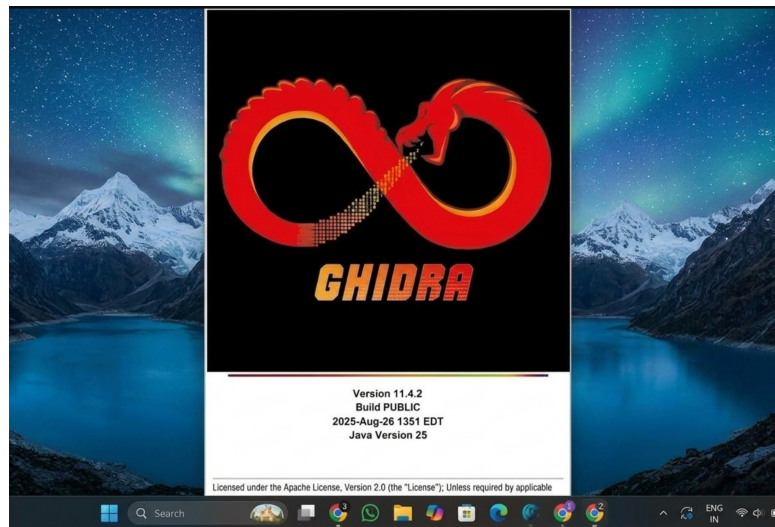
1.Repository Root

	File/Folder	Description
1	README.md	Provides an overview of Ghidra, project requirements, and tutorial structure.
2	templates/	Contains templates for documenting forensic analysis results.
3	docs/	Includes detailed, step-by-step tutorials for malware analysis.
4	scripts/	Holds automation scripts (Python/Java) to extend Ghidra’s functionality.
5	samples/	Stores safe binary or hex dump files for examination.

1. Tutorial Steps (docs/Tutorial_StepX.md)

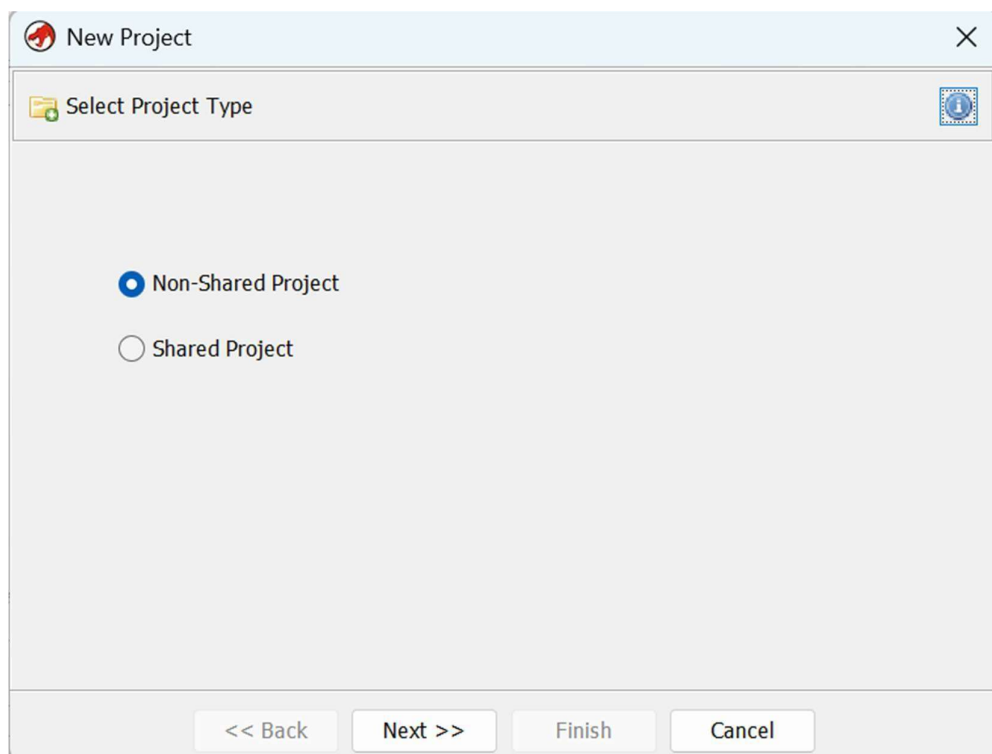
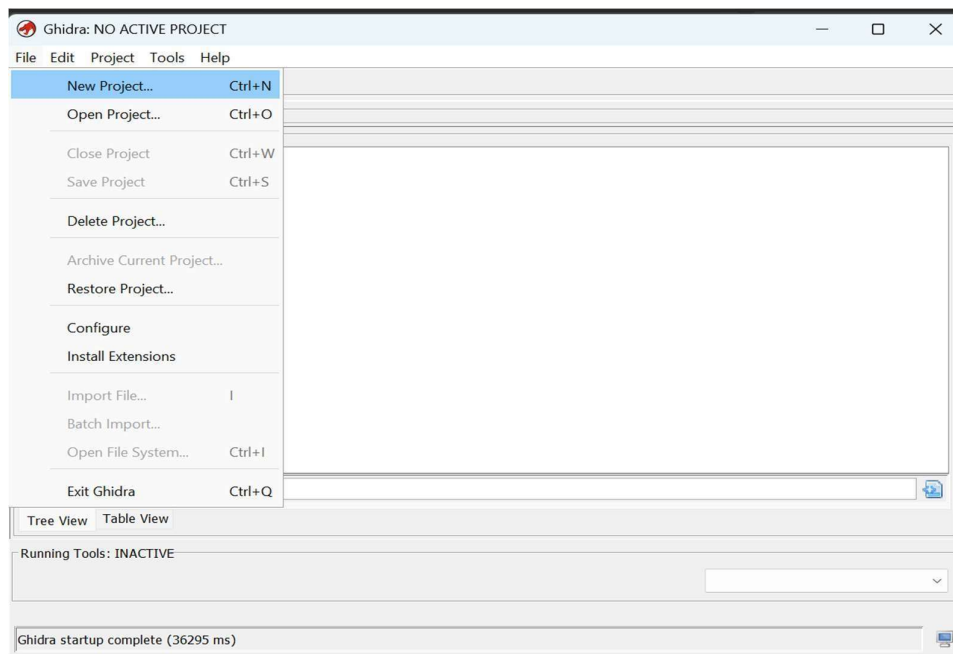
A. Environment Setup

- Set up a secure virtual machine for analysis and install Ghidra.
- Confirm the presence and compatibility of required JDK dependencies.



B. Initial Analysis

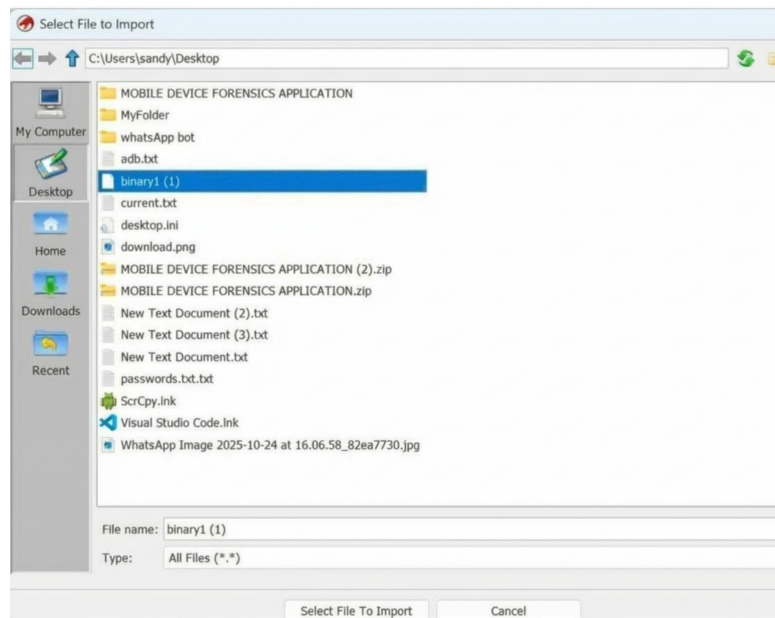
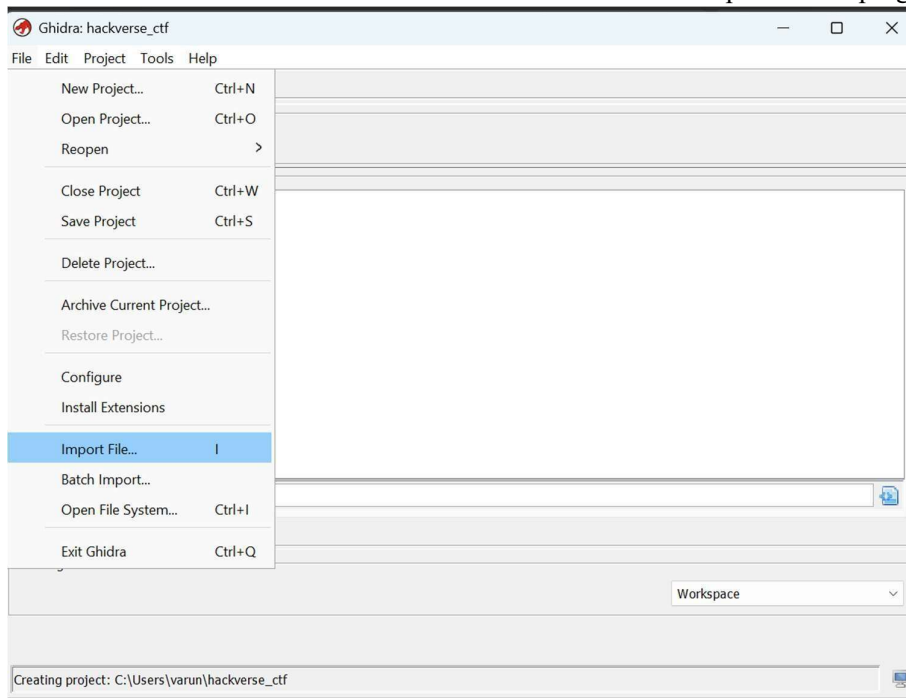
- **Loading the Binary:** Open Ghidra, create a new project, and import the target binary for analysis.
- **Auto-Analysis:** Execute Ghidra's auto-analysis feature and select the appropriate analyzers to identify functions, variables, and control flows.
- **Entry Points:** Locate the `main()` function or the initial execution point of the binary to begin examining its structure and behavior.



C. Function Analysis

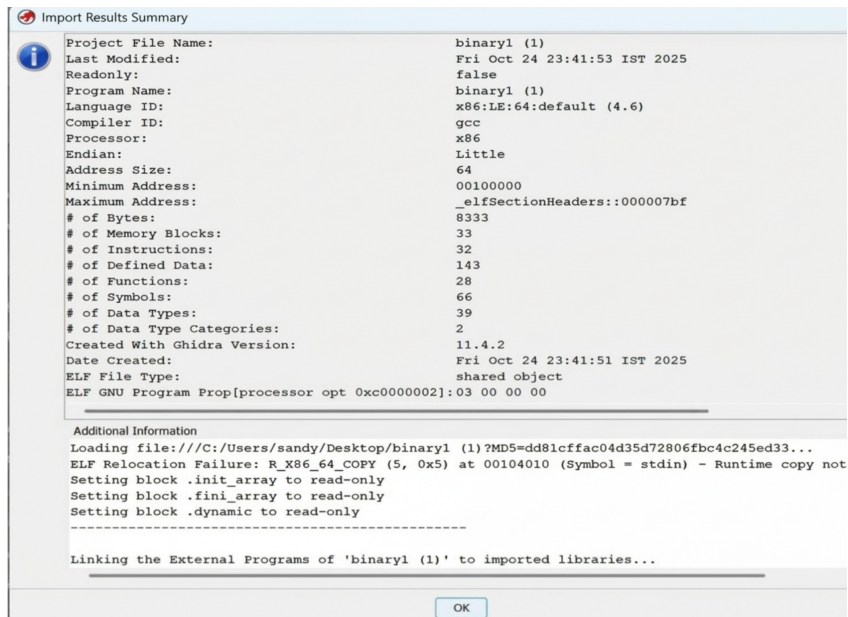
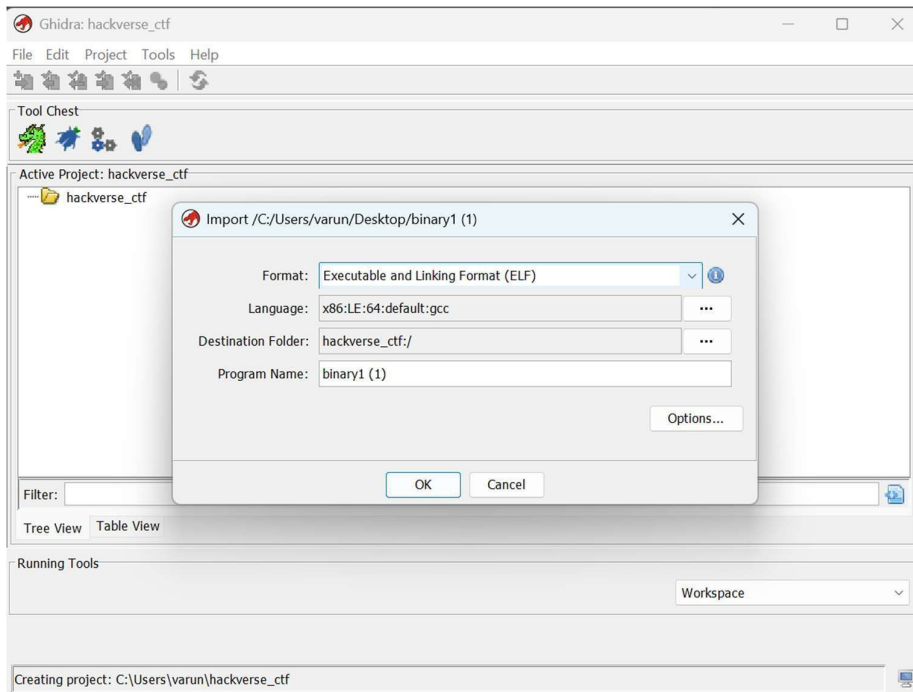
- **Decompiler:** Uses Ghidra's built-in decompiler to translate low-level assembly instructions into readable C-like code for easier understanding.
- **Function Renaming:** Rename functions based on their observed behavior to better organize and identify important operations within the binary.

- Cross-Referencing (XREF): Analyze variable references and trace function calls to understand data flow and interconnections between different parts of the program.



D. String and Import Analysis

- Strings: Examine embedded strings within the binary to identify potential indicators such as URLs, registry keys, file paths, commands, or encryption keys.
- Imports: Review imported libraries and API functions to understand the binary's external dependencies and potential behaviors, such as file access, network communication, or process manipulation.



E. Advanced Techniques (Optional)

- Control Flow Graphs (CFGs): Use Ghidra's visualization tools to map the execution flow of the binary, making it easier to trace branching logic and function calls.
- Custom Scripts: Develop automation scripts using Ghidra's Python or Java API to streamline repetitive analysis tasks, such as pattern matching or data extraction.

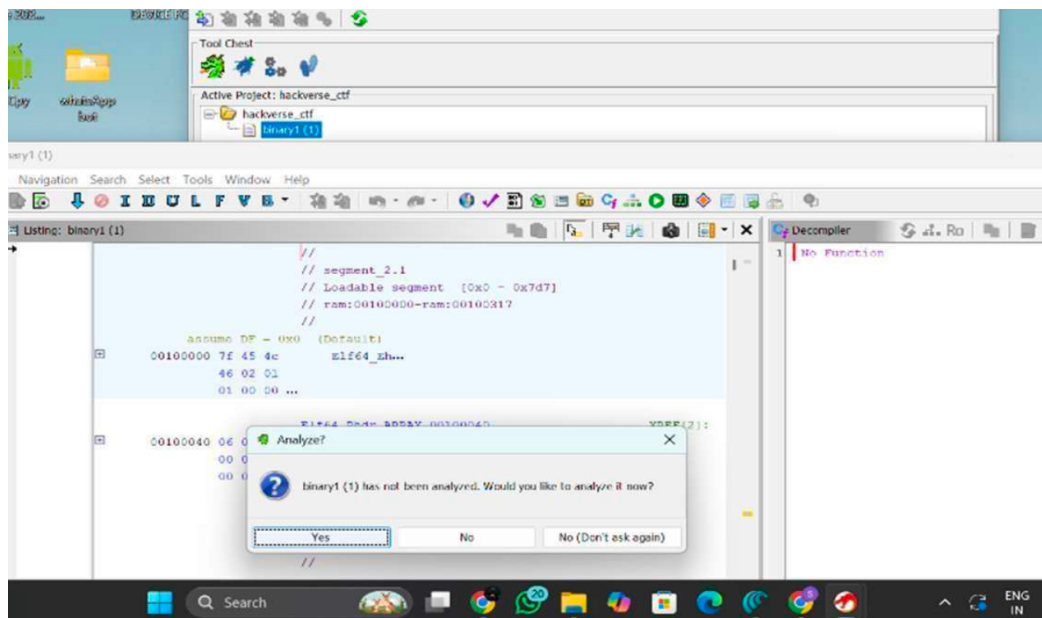
3. Example Automation Scripts (scripts/)

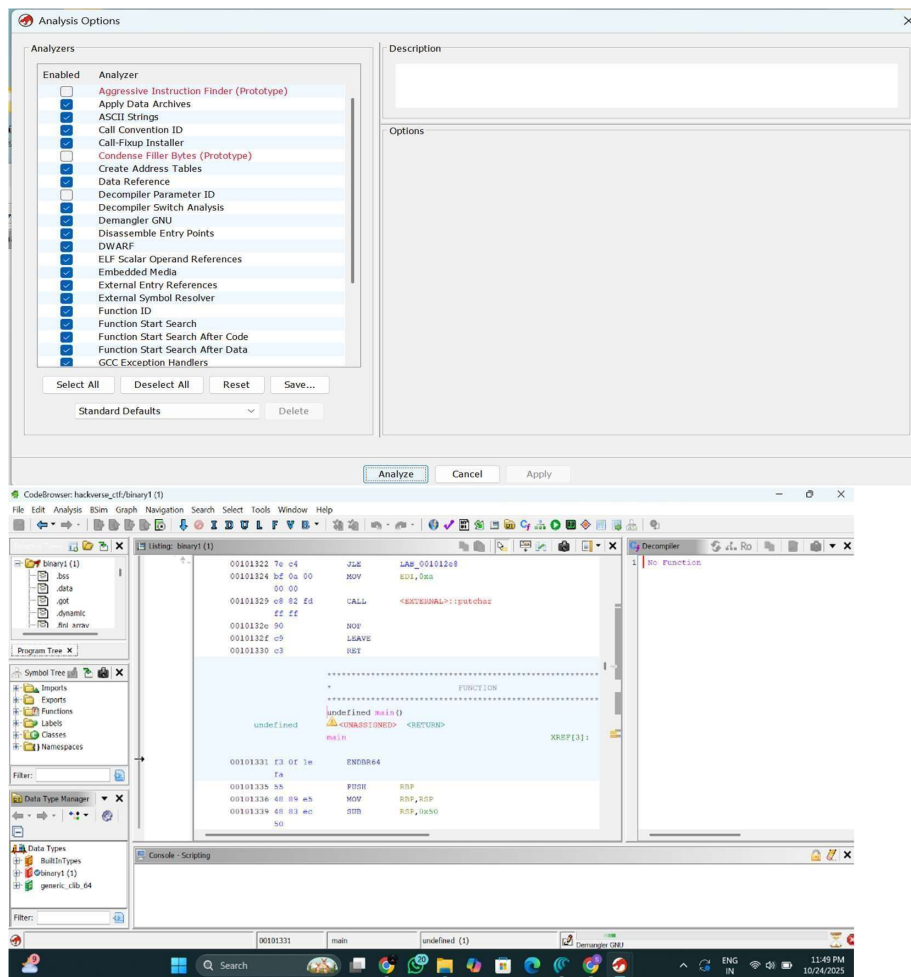
#	Script	Purpose
1	extract_strings.py	Extracts and lists all string references within the binary.
2	network_analysis.py	Identifies functions that contain network-related API calls, helping to locate potential communication routines.
3	label_functions.py	Automatically labels functions that match known malicious behavior patterns for easier identification.

4. Sample Data (samples/)

Contains safe binary files for training and practice purposes.

Example: benign_sample1.bin – A simple, harmless C program designed to simulate system-level calls and behavior for analysis demonstrations.





Result:

The malware analysis experiment using Ghidra was completed successfully. The binary was imported into a Ghidra project, auto-analysis was executed, and the code was disassembled and decompiled to C-like pseudocode. Key functions were identified and renamed, cross-references were traced, and embedded strings and imports were extracted. Control flow graphs and crossreferences clarified execution paths. Automation scripts were used to extract strings and flag network-related API calls. Artifacts and notes were documented in the repository (docs/, scripts/, samples/), and a draft report template was prepared summarizing findings and evidence. The analysis revealed indicators of persistence mechanisms, anti-analysis techniques, and potential network communication routines that merit further dynamic testing.

Conclusion:

Ghidra proved to be an effective tool for static malware analysis: it enabled clear visibility into lowlevel assembly and higher-level program behavior, allowed systematic identification of suspicious routines, and supported automation for repetitive tasks. Static analysis yielded actionable intelligence (function behavior, suspicious strings, likely persistence and I/O patterns), but some behaviors— especially runtime network activity and unpacking/anti-analysis evasion—require controlled dynamic analysis in an isolated VM to confirm. Recommended next steps are to (1) perform sandboxed

dynamic execution and network monitoring, (2) collect runtime artifacts and memory dumps, (3) document IOCs (hashes, strings, domains, registry keys) in the report, and (4) apply containment and remediation guidance based on confirmed behaviors.